Blackout Ops

當基礎設施崩潰時，仍然活著的自主維運代理

An out-of-band, fully local, self-healing SRE agent for air-gapped and degraded networks

部署：Jetson AGX Thor ・NemoClaw / OpenShell ・Nemotron-3-Nano ・TensorRT-LLM

已達成（實驗室 DGX Spark 上量測）

9/9	48 s	0	20	11	2 鑰匙
自主修復	中位數	弄更糟	白名單工具	runbook	審核代理

三項貢獻

1. 機制取代提示詞：證據先行、runbook 推送、動作前置條件全部在工具層強制，小模型也能穩定自主修復

2. 雙鑰匙審核：程式判充分條件、第二個代理判相關性與反證，破壞性動作 fail-closed

3. 算力是 Agent 管理的資源：信心分級 Tier 路由與 nvpmodel/MIG 連動，能耗成為決策對象

Operational intelligence belongs next to the infrastructure it protects.

目標場景：雲端 AI 不被允許出現的地方

Application Value 30% ·邊緣場景相關性·隱私·即時·無網

監控者不能與被監控者共命運——維運智能必須是機櫃裡的獨立裝置。

- 現行 AIOps 跑在它要保護的那座基礎設施上：叢集耗盡、儲存故障、WAN 中斷，監控一起死
- BMC / IPMI 早證明帶外管理的必要；本案把帶外從「開關機」提升到「推理與決策」
- 診斷資料就是攻擊面地圖：日誌、設定、拓撲、憑證路徑不能離開現場——本地推論是資安要求
- 常時開機的獨立裝置，真實成本是累積能耗（第 10 頁的主張由此而來）

層級	情境	雲端 AI	Blackout Ops
L1	WAN / DNS / 防火牆 / 憑證	✗	✓ 全功能
L2	金融核心、國防、半導體廠務的政策性氣隙	✗ 不准連	✓ 全功能
L3	部分網段失聯	✗	▲ 標示不可達，其餘照常

貢獻 L2 不是斷線而是不准連：一個沒有雲端替代方案的市場。本案的每一項能力都在「沒有任何對外連線」的前提下量測（離線旗標強制、CPU 上跑檢索）。

對照本次徵件：四項能力全中，四個評分項各有量測證據

Readability 10% ·評審清單直接回答

徵件要求	權重 / 門檻	本案的成就（頁）	狀態
目標與任務分解	必備	事故迴圈 Observe→Diagnose→Retrieve→Act→Verify；9 案 9/9、中位 48 s（P4）	已量·DGX Spark
工具與系統調用	至少二選	20 個白名單工具經 MCP 直接調用；HTTP ＋ MCP 雙介面（P5）	已量·DGX Spark
狀態與動態調整	至少二選	事故狀態、動作後健康推送、前置條件改路線；四層調整（P6）	已量·DGX Spark　規劃·決賽期間於 Thor
權限與風險控制	至少二選	R/W/D 分級、雙鑰匙審核、fail-closed、刻意排除清單（P7）	已量·DGX Spark
Edge Optimization	35%	TRT-LLM NVFP4 實測、prompt −51%、log 治理、檢索 28/33（P9）；能耗治理（P10）	已量·DGX Spark　規劃·決賽期間於 Thor
Agent Architecture	25%	自建決策層＋ NemoClaw 為執行基材；雙代理（指揮 / 審核）（P8）	已量·DGX Spark
Application Value	30%	氣隙市場、現場注入故障 demo、斷網後零降級（P2、P11）	已量·DGX Spark　規劃·決賽期間於 Thor
決賽要件	NeMo 深度整合	NemoClaw/OpenShell　沙　箱、Nemotron、TensorRT-LLM；Jetson Thor 部署（P8、P10）	已量·DGX Spark　規劃·決賽期間於 Thor

淘汰條款自檢：非單輪問答（每場 3–14 次工具呼叫）、非一次性生成（動作後驗證、失敗換路徑）、非純檢索（檢索只是工具之一）、非影片模擬（現場注入故障、現場斷網）。

能力一·任務分解：每一步都由上一步的證據決定

必備門檻·Agent Architecture 25%

已量·DGX Spark

成就：9 種注入故障，9/9 自主修復，中位數 48 秒，證據先行 100%，收尾驗證 100%。

```
check_health    → front / api unreachable
docker_ps       → edge-victim Exited(1) ⇐ 附上 runbook
docker_logs     → [emerg] invalid number of
                  arguments in "return" (x6)
read_config     → 看到缺分號的那一行
write_config    → written, not applied until restart
docker_restart  → health now: front 200, api 200
                  事故結束，回報並停止
```

真實演練 transcript（bad-config 案例，6 次呼叫）



- 不是把工具全跑一遍：每步的下一個動作由上一步的結果決定（Exited → 看 log；[emerg] → 讀設定；改完 → 重啟；重啟後 → 驗證）
- 修不好就回到假設生成換路徑；達不到 200/200 就帶著 exit code 與 log 升級，不空轉

貢獻 證據先行不是寫在 prompt 裡的請求，是工具層的性質：證據工具把對應 runbook 推進上下文，改變狀態的工具在沒看過 runbook 前一律拒絕。同一顆小模型，靠提示詞時六場裡一場走偏；改成機制後連續 9/9。

能力二·工具與系統調用：20 個白名單工具，全部直接可呼叫

Agent Architecture 25% ·Edge Optimization

已量·DGX Spark

成就：工具面從「24 個藏在 tool-search 迷宮後」變成「20 個直接工具」，同一顆模型從猜 tool id 變成 9/9。

類別	工具
觀察	check_health ·docker_ps ·docker_logs ·docker_exec（唯讀白名單）·read_config
主機診斷	disk_usage ·service_status ·journal_tail ·port_check ·http_check ·top_processes
知識	search_runbooks（向量檢索，11 篇）
動作	docker_start ·docker_restart ·write_config ·network_repair
破壞性	recreate_stack(replace) ·kill_process ·cleanup_disk → 審核代理
後備	self_heal（整段迴圈在主機端跑一次）

介面：ops API 同時提供 HTTP 與 MCP；沙箱內沒有 docker、沒有 systemctl，影響真實系統的路徑只有一條且都有紀錄。參數別名、JSON 字串、雙重轉義全在伺服端修正。

工具面調整前後（同一顆模型、同一組故障）

指標	NemoClaw 預設	調整後
模型直接看到的工具	3 個 meta-tool，24 個藏在後面	20 個直接工具
可見 skill	16（15 個無關）	1
System prompt 字元	28,859	14,200
每回合 tool_search	5 次	0
Heartbeat 空轉	每 30 分	關閉
提示詞要寫出指令	要，否則猜錯 id	不用，一句警報
通過 / 耗時	3/3，2.5–3 min，靠後備	9/9，中位 48 s，自主

貢獻 邊緣優化不只是壓模型，也要壓縮它必須思考的選項。工具面壓平後，六場零猜錯；之後所有失敗都追到工具層缺陷，沒有一個靠改提示詞。

能力三·狀態與動態調整：被擋下就換路，修好了就停

Agent Architecture 25%

已量·DGX Spark　規劃·決賽期間於 Thor

成就：上游脫離網路的案例，重啟前端被前置條件擋下後改走 network_repair，修復後 3/3 通過。

事故狀態（工具層維護）

- 狀態：本次事故看過的 runbook、被擋次數、證據軌跡；健康回綠燈就歸零
- 每個動作的結果自帶健康檢查：「已解決，回報並停止」或「仍失敗，哪個容器 Exited 是下一步」——修好了卻不知道，這個失敗形態從此消失
- 前置條件即時判斷：上游不在網路上時拒絕重啟前端（nginx 會解析不到而退出），並指向正確工具

四層動態調整

層	觸發 → 調整	狀態
計畫層	修復失敗 → 換假設路徑	已量·DGX Spark
模型層	信心不足 → 升 Tier（關/開 thinking）	規劃·決賽期間於 Thor
硬體層	偵測異常 → nvpmodel 拉升，結束降回	規劃·決賽期間於 Thor
通道層	網路降級 → 審批通道改本地 UI	規劃·決賽期間於 Thor

貢獻　在提示詞裡寫「修完要驗證」沒有用：模型修好後仍挖了五分鐘到 token 用完。把驗證推進動作結果之後，同一案例 201 s → 54 s。**動態調整靠的是工具回傳的狀態，不是模型的自覺。**

能力四·權限與風險控制：Self-Healing ≠ Root Shell

已量·DGX Spark

成就：破壞性動作零誤放——無證據殺行程 REJECT，量到 100% CPU 的失控行程 APPROVE，重啟後仍 500 的重建 3/3 APPROVE。

級別	內容	執行條件
R 唯讀	health ·ps ·logs ·config ·disk · journal ·port ·process	自主
W 可逆	start ·restart ·network repair · 寫回設定	自主＋審計
D 破壞性	重建容器·殺行程·清理磁碟	**雙鑰匙審核**

刻意排除：docker daemon（推論容器會跟著死，帶外裝置不能自殺）、任何系統服務、共用 image 與 volume。
其他護欄：沙箱 default-deny 只開兩個端點；探測工具只允許 loopback 與私有網段；事故軌跡永不被清理。

雙鑰匙審核代理

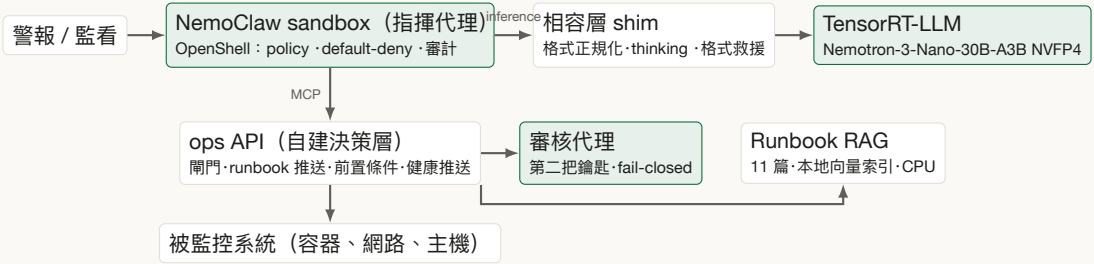
- 鑰匙一（程式）：從事故軌跡算充分條件——重建前必須重啟過且仍失敗、殺行程前 top_processes 要列到、清磁碟前用量 > 85%；缺一個直接拒絕並點名
- 鑰匙二（第二個代理，獨立 prompt、不給工具）：只判與症狀相關與否、有無一行證據反駁，要拒絕必須引用；連不上就不執行

貢獻 單靠模型審核時，同樣證據兩次判決不同（一次要求證明觀察不到的機制）。拆成兩把鑰匙後連續 3/3 一致。**確定性的條件放程式，模型只回答它答得了的問題。**每筆審核記錄事實與哪把鑰匙決定，拒絕永遠可解釋。

代理架構：自建決策層，NemoClaw 為執行基材

Agent Architecture 25% ·決賽要件·NeMo 深度整合

已量·DGX Spark



本案需求	NemoClaw / NeMo 元件
指令隔離、不得逃逸 資料絕不外流	OpenShell 沙箱 Network Policy default-deny（僅推論與 ops API）
工具受控進沙箱	MCP：20 個工具直接曝露
本地 Nemotron 推論	TensorRT-LLM PyTorch backend 直跑 NVFP4，不需 build engine

貢獻 NemoClaw 是執行基材；它不知道 [emerg] 之後該看哪個檔、不知道重啟前端會讓 nginx 退出、不知道氣隙網段裡什麼絕對禁止。**決策層（迴圈、閘門、前置條件、審核、計分卡）全是自建程式碼，攤得開。**

邊緣優化 I ·Benchmark 比對：全部離線、量測後才採用

Edge Optimization 35%

已量·DGX Spark

引擎對照（同一台、同一套腳本）

指標	vLLM 0.29 ·Qwen3.6-35B-A3B FP8 ＋ MTP	**TensorRT-LLM 1.3 · Nemotron-3-Nano NVFP4（正式採用）**
啟動到 health	275 s	**106 s**
單流 / 8 併發 tok/s	60.7 / 343	57.9 / 237
12.9k token prefill	3.5 s（cache 命中 0.3 s）	**1.17 s**
GSM8K-CoT，150 題	92.7%	**96.7%**
IFEval strict，100 題	78.0%	**80.0%**
needle 探針，8 題	8/8	5—7/8
GPU 記憶體	52 GB	**34 GB**

選型探索矩陣（vLLM，DGX Spark，32k ctx）

配置	單流 tok/s	8 併發 tok/s	needle
Qwen3-8B BF16	14.1	116.6	8/8
27B NVFP4 · ＋ MTP5 · ＋ DFlash	11.6 ·23.9 ·32.6	82 ·140 ·158	8/8
35B-A3B FP8 · ＋ MTP3 · ＋ DFlash	48.9 ·66.6 ·56.1	270 ·307 ·125	8/8

帶寬受限的機器上 MoE 小 active 是甜蜜點；探索用 vLLM，正式部署改 TensorRT-LLM ＋ Nemotron（合規、更省記憶體、prefill 快 3 倍）。

檢索方式矩陣（recall@1）

題 集：runbook 數 / 題數	關鍵字	**向量**	混合
8 / 20	15	**20**	18
9 / 23	16	**23**	20
11 / 33	19	**28**	23
權 重 掃 描，8 / 20	向量 20 ·關鍵字權重 0.1 得 20 ·0.2—0.5 得 19 ·RRF 18 ·max 17		

輸入端治理

指標	前	後
System prompt 字元	28,859	14,200（—51%）
crash loop 案例（log 截斷修正）	0/5	4/4
每次事故 token	中位約 2.7k，區間 1.5k—10k	

貢獻 結論全部與直覺相反才值得放：混合檢索輸給純向量、關 block reuse 更快（首個長 prompt 18.7 s → 1.2 s）、NGram 投機解碼會炸、PDL 無感、prompt 越短工具越準。

邊緣優化 II ·量測計畫：算力不是限制條件，是 Agent 自己要管理的資源

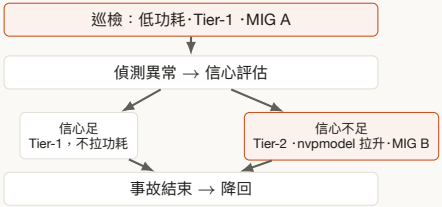
Edge Optimization 35% ·初賽給方法與對照組，決賽於 Jetson AGX Thor 交數字

規劃·決賽期間於 Thor　已量·DGX Spark

全場每隊都拿到 128 GB；帶外裝置的真實成本是累積能耗，所以量的是「何時花、花多少、會不會排擠監控」。

機制

- Tier-1 = 同一顆 30B A3B 關 thinking；Tier-2 = 開 thinking。已量：token 幾乎全在 thinking，第一刀切在這裡
- 信心不足 → 升 Tier-2 同時 nvpmodel 拉升；MIG 把巡檢與推理分開
- 輸入端：四級日誌預處理，量壓縮比 vs 正確率



量測計畫矩陣（每個指標：怎麼量、跟誰比、什麼算通過）

指標	量測方法	對照組	通過標準
單次診斷能耗 Wh	tegrastats 200 ms 取樣積分	同機、全程 Tier-2 固定高功耗	下降，且正確率不降
24 h 累積能耗 Wh	同上，含巡檢時段	1U 監控伺服器 (牆上電表)	低於基準
功耗動態範圍（倍）	巡檢模式 vs 推理模式平均功耗	—	報告倍率，取代「低功耗」形容詞
Tier-1 承接比例	計分卡逐案記錄	—	越高越好，正確率不降
每瓦診斷次數	診斷次數 ÷ Wh	x86 工作站＋獨顯（牆上電表）	高於基準；量測邊界分開標示
壓縮比 vs 正確率	4 級預處理 × 9 案 × 計分卡	無壓縮	畫曲線，標拐點與工作點
巡檢不被推理餓死	MIG 實例 A 的健康輪詢 p99	無 MIG 共用 GPU	p99 不惡化

目前已量的依據（DGX Spark）

每次事故 token 中位 2.7k、區間 1.5k–10k，幾乎全在 thinking → Tier 切在 thinking 開關；prompt −51% 與日誌折疊已上線，會直接反映在能耗指標上；配發清單裡的 2U 寬壓工業機型即為部署形態。

貢獻　對照組是同一台機器自比，控制變因乾淨；x86 基準對的是現行 AIOps 真正跑的環境。同一段程式碼同時是能耗優化（35%）與 Agent 的自主資源決策（25%）。

評估矩陣：我們量的不是回答，是系統有沒有恢復

Application Value 30% ·現場驗證

已量·DGX Spark　規劃·決賽期間於 Thor

九案逐案計分（最新一輪，所有修正就位）

案例	結果	秒	呼叫	效率	token
nginx-stopped	自主	44	4	1.00	2.0k
bad-config	自主	79	6	1.00	4.7k
upstream-down	自主	36	4	1.00	1.5k
bad-upstream-name	自主	54	7	0.67	2.7k
container-removed	自主	30	3	1.00	1.8k
crashloop	自主	123	6	1.00	6.5k
upstream-paused	自主	46	3	1.00	1.9k
upstream-detached	自主	206	14	0.50	10.2k
upstream-500	自主	48	6	0.50	2.7k
合計 / 中位數	9/9	48	6	0.85	2.7k
當天全部 86 場（含修正前）	73 自主	弄更糟 1 ·假健康宣稱 0 ·誠實升級欄待 unseen 案例			

效率 = 該案最少動作數 ÷ 實際執行的狀態改變動作；upstream-500 為 0.50 是 runbook 規定先重啟再重建。計分卡另量證據先行、收尾驗證、root cause 點名、無關與重複呼叫、閘門與審核擋下次數。

現場 demo（約 6.5 分鐘，不預錄）



結果分級：自主 / 靠後備 / 誠實升級 / 弄更糟 / 沉默失敗——單一通過率看不見前三種，這張卡各有一欄。

貢獻　失敗分析方法本身是成果：每個失敗都從沙箱裡模型的 thinking 追根因（log 截斷砍掉 stderr 的 [emerg]、上游不在網路時重啟前端、把 nginx 的 read-only 提示當成改不了），修的都是工具。**可重現的故障 harness ＋計分卡，讓改進是量出來的。**

貢獻、既有成果揭露、決賽交付

貢獻（新做出來的）

1. 工具層強制的證據先行：runbook 推送＋動作閘門＋前置條件，機制取代提示詞
2. 雙鑰匙審核代理：程式判充分條件、第二代理判相關與反證，fail-closed
3. 全地端 NemoClaw ＋ TensorRT-LLM 維運代理，沙箱內 20 個工具經 MCP 直接調用
4. 可重現的故障 harness（9 案）與多維計分卡，改進可量測
5. 算力當作 Agent 管理的資源：信心分級 Tier 路由與 nvpmodel/MIG 連動（決賽交付）

既有成果揭露（規則要求）

2026/09/21–09/29 於實驗室 DGX Spark 建置並量測：NemoClaw/OpenShell 沙箱＋ TensorRT-LLM 服務、20 個白名單工具的 ops API（HTTP ＋ MCP）、11 篇 runbook 與向量檢索、9 種故障注入與 harness、雙鑰匙審核代理、演練計分卡。

決賽期間新增

Jetson Thor 部署、nvpmodel/MIG 能耗治理與表一表二、信心分級 Tier 路由、日誌壓縮曲線、本地審批 UI、未演練過的故障案例（unseen 通過率）

風險與備案

Thor 上 TensorRT-LLM 可用性 → NIM / Ollama 基準線可 demo

配發 OEM 機型核心模組 → 工作坊當天優先驗證

Privacy Router 無法程式切換 → Tier 路由做在自有相容層

功耗動態範圍不如預期 → 日誌壓縮為第二條腿

配發偏好：Jetson AGX Thor 開發者套件 > ASUS PE3000N > Aetina AIE-KT78

OBSERVE LOCALLY.　REASON LOCALLY.　RECOVER LOCALLY.

Blackout Ops —a fully local autonomous SRE agent for resilient edge infrastructure